Árvores de decisão e redes neurais em duas tarefas de classificação: alvos proteicos do ChEMBL e aprovação de empréstimos

Bernan R. do Nascimento[1], **Paulo E. R. Araujo**[1], **Thiago R. P. de Carvalho**[1]

[1]Universidade Federal do Piauí (UFPI)
Programa de Pós-Graduação em Ciência da Computação – Teresina – PI – Brasil
PPGCC010 – Aprendizagem de Máquina (2026.2) – Prof. Vinicius Ponte Machado

bernanr7@gmail.com, pauloaraujo@ufpi.edu.br

thiago.rafael.pc@gmail.com

Abstract. *Predicting which protein target a molecule is associated with is an early step in drug discovery. We rebuild from the ChEMBL 37 API a dataset of 27,714 molecules, each associated with one of four targets (EGFR, BACE1, DRD2 and HDAC1), and train a decision tree on five physicochemical descriptors and twelve counts taken from the SMILES string, a text that describes a molecule's structure. With a grouped split, the SMILES counts raise accuracy from 77.1% to 89.4% (a 16% relative gain; mean of 10 seeds). A random split overestimates accuracy by up to 5.3%, because a quarter of the test molecules have a twin with identical descriptors in the training set; the grouped split keeps such twins on the same side. Weka's Random Tree, which samples the features tried at each node, falls 0.4% below the decision tree, while tree ensembles gain 6.0% to 8.1%: feature sampling hurts a single tree but, across 300 trees, it is what makes the Random Forest work. On a second dataset, 4,269 loan applications, a multilayer perceptron reaches 96.4%, above a one-threshold rule (95.5%) and logistic regression (91.9%) but below a decision tree (98.1%), which also trains about 50 times faster. The labels follow a step on the credit score: the tree draws it with one cut, while the network approximates it with a ramp, where its errors concentrate.*

Resumo. *Prever a qual proteína-alvo uma molécula está associada é uma etapa inicial da descoberta de fármacos. Reconstruímos pela API do ChEMBL 37 uma base de 27.714 moléculas, cada uma associada a um de quatro alvos (EGFR, BACE1, DRD2 e HDAC1), e treinamos uma árvore de decisão com cinco descritores físico-químicos e doze contagens extraídas do SMILES, texto que descreve a estrutura da molécula. Com divisão agrupada, as contagens elevam a acurácia de 77,1% para 89,4% (ganho relativo de 16%; média de 10 seeds). A divisão aleatória superestima o resultado em até 5,3%, pois um quarto das moléculas de teste tem uma gêmea de descritores idênticos no treino; a divisão agrupada mantém essas gêmeas do mesmo lado. A Random Tree do Weka, que sorteia os atributos de cada nó, fica 0,4% abaixo da árvore, enquanto conjuntos de árvores ganham de 6,0% a 8,1%: o sorteio de atributos piora uma árvore sozinha, mas, somado em 300 árvores, é o que faz a Random Forest funcionar. Numa segunda base, de 4.269 pedidos de empréstimo, uma rede neural (perceptron de*

múltiplas camadas) acerta 96,4%, acima de uma regra de um corte (95,5%) e da regressão logística (91,9%), mas abaixo da árvore de decisão (98,1%), que ainda treina cerca de 50 vezes mais rápido. Os rótulos seguem um degrau no score de crédito: a árvore o representa com um corte, e a rede o aproxima com uma rampa, onde se concentram seus erros.

1. Introdução

Na descoberta de fármacos, saber a qual proteína uma molécula está associada ajuda a priorizar experimentos. Bancos abertos como o ChEMBL [Gaulton et al. 2012, Zdrazil et al. 2024] reúnem milhões de medições de potência de moléculas contra proteínas, e com eles é possível treinar modelos que sugerem o alvo de uma molécula [Keiser et al. 2007].

Este trabalho trata uma versão controlada do problema: dada uma molécula associada a exatamente uma de quatro proteínas (EGFR, BACE1, DRD2 e HDAC1), prever qual é. A molécula é descrita por números calculados a partir dela: cinco descritores físico-químicos e contagens extraídas do SMILES, um texto que descreve a estrutura da molécula. O modelo principal é uma árvore de decisão [Breiman et al. 1984], escolhida porque suas regras podem ser lidas e discutidas. Depois, ela é comparada com a Random Tree, uma árvore que sorteia os atributos de cada nó, e com conjuntos de árvores.

Três pontos tornam o problema menos trivial do que parece. Primeiro, o conjunto do Kaggle que inspirou o trabalho está incompleto e foi preciso reconstruí-lo do ChEMBL. Segundo, cinco descritores globais separam mal as classes. Terceiro, muitas moléculas compartilham exatamente os mesmos descritores: se uma cai no treino e outra no teste, o modelo é premiado por memorizar. Por isso comparamos a divisão aleatória, que sorteia as moléculas livremente, com a divisão agrupada, que mantém do mesmo lado as moléculas de descritores idênticos (Seção 5.2). As contribuições são:

- uma base reconstruída do ChEMBL_37, com 27.714 moléculas de quatro alvos e curadoria documentada passo a passo;
- 12 atributos obtidos apenas contando caracteres do SMILES, que, somados aos descritores, elevam a acurácia da árvore em 16% (ganho relativo);
- uma avaliação com divisão agrupada e 10 seeds, que mede o quanto a divisão aleatória superestima o resultado (de 1,5% a 5,3%, em termos relativos);
- uma comparação da árvore com a Random Tree do Weka, a Random Forest e o *gradient boosting*, que mostra por que o sorteio de atributos, que piora uma árvore sozinha, é a peça da Random Forest;
- numa segunda base, de pedidos de empréstimo, uma comparação de uma rede neural com a árvore e com modelos de referência, incluindo o tempo de treino, que mostra quando a rede não compensa (Seção 7).

Na última parte, o problema muda de domínio: prever se um pedido de empréstimo foi aprovado, a partir de 11 atributos do pedido e de quem o fez. A base é pequena, sem valores ausentes, e o rótulo depende quase só do score de crédito. Ela serve para estudar o treino de uma rede neural (padronização, número de neurônios, épocas e parada antecipada) e para comparar a rede com modelos mais simples num caso em que a forma da regra que gerou os rótulos é conhecida.

2. Fundamentação e trabalhos relacionados

Predição de alvos. Bancos como o ChEMBL [Gaulton et al. 2012, Zdrazil et al. 2024] reúnem medições de potência de moléculas contra alvos proteicos. Uma abordagem clássica para sugerir o alvo de uma molécula compara conjuntos de moléculas por similaridade química e relaciona proteínas pela semelhança do que elas ligam, o que permitiu prever interações novas depois confirmadas em laboratório [Keiser et al. 2007]. Este trabalho segue outra linha, mais simples e interpretável: classificar a molécula a partir de atributos numéricos calculados dela.

Descritores e SMILES. A regra dos cinco de Lipinski [Lipinski et al. 2001] usa massa molecular, LogP e os números de doadores e aceptores de ligação de hidrogênio, propriedades calculadas direto da estrutura da molécula. A TPSA [Ertl et al. 2000] é a área polar da molécula, estimada somando contribuições de fragmentos. O SMILES [Weininger 1988] descreve a molécula como texto: letras são átomos, dígitos fecham anéis e parênteses abrem ramificações. Contar esses símbolos dá atributos de composição sem analisar a estrutura, ao custo de não enxergar subestruturas, isto é, como os átomos estão ligados entre si.

Árvores de decisão. O CART [Breiman et al. 1984] divide o espaço de atributos de forma recursiva, em cada nó escolhendo o limiar de um atributo que mais reduz a impureza (Gini ou entropia). Sem limite de profundidade, a árvore acerta todo o treino; a poda por custo-complexidade, controlada por α, corta ramos que pouco reduzem o erro e diminui o número de folhas. A vantagem é a legibilidade das regras.

Random Tree e conjuntos de árvores. A Random Tree do Weka [Witten et al. 2016] é uma árvore única que, em cada nó, sorteia K dos n atributos e procura o melhor corte só entre eles; o padrão é $K = \lfloor \log_2 n \rfloor + 1$, sem poda. Sozinha, ela é uma árvore pior, porque o sorteio pode esconder o melhor corte. O sorteio existe para os conjuntos. A Random Forest [Breiman 2001] treina muitas árvores com sorteio de atributos em cada nó, cada uma numa amostra com reposição do treino (*bootstrap*), e a previsão é a média dos votos; o sorteio torna as árvores menos parecidas entre si e a média mais eficaz. O *gradient boosting* [Friedman 2001] também soma árvores, mas em sequência: cada árvore nova corrige o erro que as anteriores ainda cometem. A versão por histogramas [Ke et al. 2017] agrupa cada atributo em faixas antes de procurar cortes, o que acelera o treino. Os conjuntos costumam ser mais precisos que uma árvore única, ao custo de não terem regras legíveis.

Vazamento e validação. Vazamento de dados, quando informação do teste chega ao treino, é uma causa documentada de resultados superestimados em ciência baseada em aprendizado de máquina [Kapoor and Narayanan 2023]. Em química, a divisão aleatória já é conhecida por ser otimista; Sheridan [Sheridan 2013] mostra que separar treino e teste por data estima melhor o desempenho prospectivo. Aqui o vazamento é mais direto: moléculas distintas com os mesmos cinco descritores podem cair em lados opostos da divisão. Por isso o teste agrupa as moléculas pela tupla de descritores (Seção 5.2).

Redes neurais. Um neurônio artificial calcula uma soma ponderada das entradas, mais um viés, e aplica a ela uma função de ativação. Com a sigmoide, que leva qualquer número a um valor entre 0 e 1, um único neurônio é a regressão logística. O perceptron de múltiplas camadas (MLP) empilha camadas de neurônios entre a entrada

e a saída, as camadas ocultas; com a ativação ReLU, $\max(0, z)$ [Nair and Hinton 2010], elas permitem fronteiras de decisão não lineares. Os pesos começam sorteados e são ajustados pela retropropagação do erro [Rumelhart et al. 1986]: a cada lote de exemplos, calcula-se quanto cada peso contribuiu para a perda (aqui, a entropia cruzada) e move-se o peso no sentido de reduzi-la, com um passo dado pela taxa de aprendizado. O otimizador Adam [Kingma and Ba 2015] adapta esse passo para cada peso. Uma época é uma passada completa pelo treino. Como o passo é o mesmo para entradas de escalas muito diferentes, os atributos costumam ser padronizados antes [LeCun et al. 1998]. Treinar por épocas demais leva a rede a se ajustar a detalhes do treino; a parada antecipada [Prechelt 1998] separa uma parte do treino como validação e interrompe o treino quando o desempenho nela para de melhorar. Em dados tabulares, modelos de árvore ainda costumam superar redes neurais, em parte porque as redes tendem a aprender funções suaves e as árvores representam bem relações irregulares, como degraus [Grinsztajn et al. 2022].

3. Dados e curadoria

3.1. Base de partida

O trabalho partiu do conjunto *MultiTarget Bioactivity ChEMBL*, publicado no Kaggle [xjoannax88 2026]. O arquivo disponível, porém, não corresponde à descrição da própria página: tem 1.333 linhas em vez de 8.711, nenhuma molécula de EGFR, apenas duas de BACE1 e 95,5% das linhas em DRD2. A página também associa BACE1 ao identificador CHEMBL1987, que designa outra proteína; o correto é CHEMBL4822. Como as versões anteriores exigem login para download, a base foi reconstruída diretamente do ChEMBL.

3.2. Reconstrução a partir do ChEMBL

Os dados vêm da API REST do ChEMBL [Davies et al. 2015], um banco curado de moléculas e de suas medições de potência [Gaulton et al. 2012, Zdrazil et al. 2024], na versão ChEMBL_37 (lançada em 01/05/2026, extração em 24/10/2026). Para cada um dos quatro alvos da Tabela 1 foram baixadas as medições de ensaios de ligação com valor exato (sem desigualdades) e com pChEMBL preenchido. O pChEMBL é uma medida de potência padronizada pelo ChEMBL ($-\log_{10}$ de uma concentração molar): quanto maior, mais potente a molécula contra o alvo. Neste trabalho, uma molécula é associada a um alvo quando seu pChEMBL contra ele é de pelo menos 6, o que equivale a 1 μM.

A Figura 1 resume a curadoria. As 58.736 medições brutas perdem 7.328 registros marcados pelo ChEMBL como duplicata potencial e 49 sem SMILES. As 51.359 restantes são agregadas pela mediana do pChEMBL em cada par molécula-alvo (37.424 pares). Mantêm-se os pares com pChEMBL ≥ 6 (27.798) e removem-se 42 moléculas associadas a mais de um alvo, cujo rótulo seria ambíguo. O resultado são 27.714 moléculas, uma por linha, com a proteína-alvo como classe.

Cada molécula é descrita pelos cinco descritores que o ChEMBL calcula para ela: massa molecular, LogP, números de aceptores e doadores de ligação de hidrogênio (HBA e HBD) e área de superfície polar topológica (TPSA) [Ertl et al. 2000]. O pChEMBL não é usado como atributo, pois resulta do ensaio contra o próprio alvo e vazaria o rótulo. Nas 525 moléculas presentes também no arquivo do Kaggle, 99,6% têm o mesmo rótulo e 99,4% os mesmos descritores.

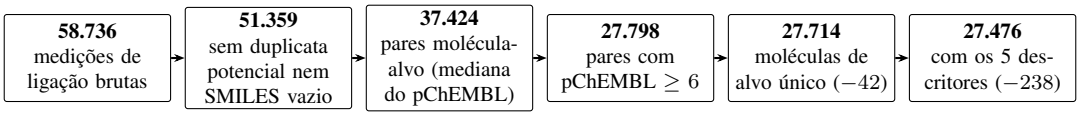


Figura 1. Funil de curadoria, do download à base de modelagem.

Tabela 1. Classes da base final. A última coluna é a proporção das 27.714 moléculas.

Alvo	ChEMBL ID	Moléculas	%
EGFR	CHEMBL203	8.108	29,3
BACE1	CHEMBL4822	7.344	26,5
DRD2	CHEMBL217	6.650	24,0
HDAC1	CHEMBL325	5.612	20,2

3.3. Problemas conhecidos

Há duas ressalvas. (i) Em 238 moléculas (0,86%) faltam LogP, HBA, HBD e TPSA, sobretudo em moléculas muito grandes; essas linhas foram removidas, restando 27.476 para modelagem. (ii) 4.777 linhas repetem exatamente os cinco descritores de outra linha, sempre da mesma classe; isso é a origem do vazamento tratado nas Seções 5.2 e 6.1.

O código, os notebooks e os dados estão em `https://github.com/pumba-dev/machine-learning-practice`. Os dados derivados do ChEMBL seguem a licença CC BY-SA 3.0.

4. Análise exploratória

As classes são razoavelmente balanceadas (Tabela 1): responder sempre EGFR, a majoritária, acerta 29,3%, e esse é o piso de comparação do restante do trabalho. A Figura 2 compara os cinco descritores entre os alvos. DRD2 tem TPSA e HBD claramente menores que os demais. EGFR tem os maiores valores de massa molecular, LogP e HBA. HDAC1 tem o maior HBD. BACE1 e HDAC1 se sobrepõem em massa molecular, TPSA e HBA.

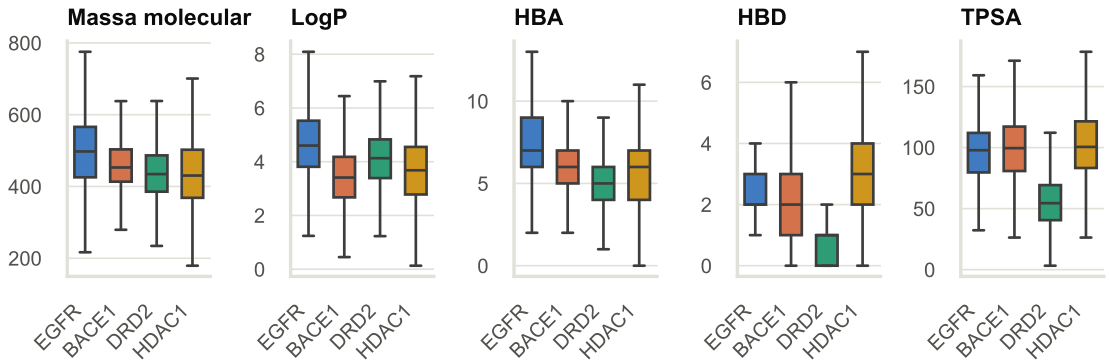


Figura 2. Descritores por proteína-alvo. Caixas mostram os quartis e as hastes 1,5 vez a amplitude interquartil; valores extremos foram omitidos.

TPSA, HBA e HBD são correlacionadas entre si (correlação de Spearman de 0,72

entre TPSA e HBA e de 0,69 entre TPSA e HBD). Isso não prejudica a árvore, mas divide a importância entre elas.

Para medir quanto cada descritor separa os alvos sozinho, fizemos um experimento: para cada um dos cinco, treinamos uma árvore de decisão de profundidade 3 que enxerga apenas aquele atributo e medimos sua acurácia por validação cruzada de 5 partes. Os resultados foram 37,8% (massa molecular), 38,7% (LogP), 43,3% (HBA), 44,9% (TPSA) e 45,1% (HBD). Todos superam os 29,3% de responder sempre a classe majoritária, mas nenhum passa de 45,1%. Nenhum descritor separa os alvos sozinho, então a árvore precisa combiná-los, e mesmo assim o desempenho esperado com só cinco descritores globais é limitado, sobretudo entre BACE1 e HDAC1. Isso motiva os atributos extraídos do SMILES (Seção 5).

5. Metodologia de modelagem

Tudo foi implementado em Python com scikit-learn [Pedregosa et al. 2011]. A tarefa é classificação multiclasse: prever a proteína-alvo (4 classes) das 27.476 moléculas com os cinco descritores completos.

5.1. Conjuntos de atributos

Além dos descritores, foram criados 12 atributos por contagem de caracteres do SMILES: comprimento da cadeia; número de átomos de N, O e S; de halogênios (F, Cl, Br, I); de anéis (dígitos de fechamento dividido por 2); de ramificações (`(`); de átomos aromáticos (c, n, o, s minúsculos); de ligações duplas (=) e triplas (#); de cargas (+, −); e de marcas de estereoquímica (`@`, `/`). Comparam-se três conjuntos de atributos: os 5 descritores, as 12 contagens de SMILES e os 17 atributos juntos.

5.2. Divisão treino-teste e vazamento

Os dados são divididos em treino (80%, 21.980 moléculas) e teste (20%, 5.496). Há um detalhe: moléculas diferentes podem ter exatamente os mesmos cinco valores de descritores (4.777 linhas repetem outra, sempre da mesma classe). Para o modelo, elas são o mesmo ponto. Se uma cai no treino e a outra no teste, acertar a segunda não exige generalizar: basta lembrar a primeira. Comparamos duas formas de dividir.

Na *divisão aleatória*, sorteia-se 20% das moléculas para o teste, mantendo a proporção de cada classe e sem olhar as repetições: moléculas com descritores idênticos podem ficar em lados opostos. Na *divisão agrupada*, primeiro reúnem-se num mesmo grupo todas as moléculas com os mesmos cinco descritores, e depois sorteiam-se grupos inteiros para o teste, também mantendo a proporção de classes: um grupo nunca é partido entre treino e teste. Na prática, usa-se uma validação cruzada de 5 partes, estratificada por classe e sem separar grupos, e uma das partes (20%) serve de teste. A diferença de acurácia entre as duas divisões mede o vazamento [Kapoor and Narayanan 2023].

A combinação dos 3 conjuntos de atributos com as 2 divisões dá os 6 cenários da Tabela 2.

5.3. Ajuste e avaliação da árvore de decisão

Em cada cenário da Tabela 2, uma busca em grade com validação cruzada de 5 partes escolhe os hiperparâmetros da árvore de decisão: critério de impureza (Gini ou entropia),

Tabela 2. Os seis cenários: cada conjunto de atributos com cada divisão treino-teste.

Cenário	Atributos	Divisão
C1	5 descritores	aleatória
C2	5 descritores	agrupada
C3	12 contagens de SMILES	aleatória
C4	12 contagens de SMILES	agrupada
C5	17 (descritores + contagens)	aleatória
C6	17 (descritores + contagens)	agrupada

profundidade máxima (8, 12, 16 ou sem limite), mínimo de amostras por folha (1, 5 ou 20) e parâmetro de poda por custo-complexidade α (0, 10^{-4} ou 3×10^{-4}), 72 combinações. A validação interna segue a mesma regra da divisão externa: na divisão agrupada, as partes também não separam grupos. A métrica de escolha é a acurácia. No teste, reportam-se acurácia, F1 macro (média simples do F1 das quatro classes), F1 por classe e a matriz de confusão. Como piso, usa-se um classificador que responde sempre a classe majoritária (EGFR).

5.4. Random Tree e conjuntos de árvores

Os demais modelos usam o mesmo treino, o mesmo teste e a mesma validação cruzada agrupada da árvore. A Random Tree é implementada como uma árvore do scikit-learn com sorteio de atributos em cada nó (`max_features`), o que reproduz o algoritmo do Weka. Ela é avaliada de duas formas nos seis cenários: com os padrões do Weka (entropia, $K = \lfloor \log_2 n \rfloor + 1$, sem poda, o que dá $K = 5$ com 17 atributos) e com uma busca em grade de 54 combinações (critério, fração de atributos sorteados de 0,25, 0,5 ou 0,75, profundidade máxima e mínimo por folha).

A Random Forest e o *gradient boosting* por histogramas são avaliados no cenário principal (17 atributos, divisão agrupada), com os padrões do scikit-learn e com uma busca leve: para a floresta, de 300 árvores, a fração de atributos sorteados e o mínimo por folha (6 combinações); para o *boosting*, com 300 iterações, a taxa de aprendizado e o número máximo de folhas por árvore (9 combinações). Dois recursos ficam desligados porque separam uma validação aleatória dentro do treino e deixariam gêmeas dos dois lados: a estimativa *out-of-bag* da floresta e a parada antecipada do *boosting*.

Para ver o efeito do sorteio de atributos numa árvore só e num conjunto, três modelos com a mesma árvore base, sem poda, são comparados na seed 42: a árvore de decisão (sem sorteio), a Random Tree com $K = \sqrt{n}$, o padrão da floresta, e a Random Forest de 300 dessas árvores. As duas árvores únicas usam a média de 10 sementes, porque a Random Tree varia com a semente.

5.5. Repetição com várias seeds

Um único sorteio pode ser favorável ou desfavorável por acaso. Todo o procedimento acima, com todos os modelos, é repetido com 10 seeds (0 a 9), que mudam as duas divisões, as partes da validação cruzada e a semente dos modelos. As tabelas trazem média $\pm$ desvio padrão amostral. Ganhos e superestimativas são relativos, $(B - A)/A$,

calculados em cada seed e depois resumidos pela média; a superestimativa compara a acurácia da divisão aleatória (B) com a da agrupada (A). As curvas, a árvore desenhada, a comparação entre as três árvores base e a análise de erros usam a seed 42. A variação das seeds cobre o sorteio da divisão, mas não a escolha dos dados: a base é a mesma em todas as repetições.

6. Resultados e discussão

Salvo indicação em contrário, os números são média ± desvio padrão em 10 seeds (Seção 5.5); as figuras e os exemplos de erro usam a seed 42. Acurácias e F1 estão em %; ganhos e superestimativas são relativos.

6.1. Vazamento por descritores repetidos

Das 27.476 moléculas, 22.699 têm uma tupla de descritores distinta e 4.777 repetem a de outra molécula (o maior grupo tem 12). Nenhum grupo mistura classes. Na divisão aleatória, 25,8 ± 0,4% das moléculas de teste têm uma gêmea de descritores no treino, isto é, outra molécula com exatamente os mesmos cinco valores. O efeito aparece na última coluna da Tabela 3: com só os cinco descritores, a divisão aleatória superestima a acurácia em 5,3 ± 1,2%, pois a árvore pode decorar as gêmeas. Com as contagens de SMILES o excesso cai para 1,5% tanto com elas sozinhas quanto com os 17 atributos, porque moléculas com os mesmos descritores têm SMILES diferentes e as contagens não são copiadas junto. A partir daqui, usa-se a divisão agrupada.

Tabela 3. Árvore de decisão no teste, por conjunto de atributos (média ± desvio em 10 seeds). Cada linha reúne dois cenários da Tabela 2. Superestimativa: quanto a acurácia da divisão aleatória excede a da agrupada, em termos relativos.

Atributos	Acurácia aleatória (%)	Acurácia agrupada (%)	F1 macro agrupado (%)	Superestimativa (%)
Descritores (5)	81,2 ± 0,5	77,1 ± 0,8	77,0 ± 0,8	5,3 ± 1,2
SMILES (12)	88,9 ± 0,4	87,6 ± 0,6	87,3 ± 0,6	1,5 ± 1,0
Ambos (17)	90,8 ± 0,3	89,4 ± 0,4	89,3 ± 0,4	1,5 ± 0,4

6.2. Efeito dos atributos de SMILES

Na divisão agrupada, os cinco descritores dão acurácia de 77,1 ± 0,8%. As 12 contagens de SMILES sozinhas chegam a 87,6 ± 0,6%, um ganho relativo de 14%. Os 17 atributos juntos chegam a 89,4 ± 0,4% (F1 macro de 89,3%), um ganho relativo de 16% sobre os descritores, muito acima da variação entre seeds. Contar letras de uma cadeia de texto carrega mais informação sobre a molécula, como o número de anéis aromáticos e de halogênios, do que cinco propriedades globais. Na seed 42, a árvore escolhida acerta 89,7%, dentro da faixa de 88,6% a 89,9% observada nas 10 seeds.

6.3. Profundidade e poda

A busca em grade escolheu a entropia como critério e nenhum limite de profundidade nos seis cenários, e só ativou a poda quando os atributos eram apenas os descritores: com cerca de 22 mil linhas de treino, a regra de parada da própria árvore já fica perto do

ideal. A Figura 3a mostra o motivo. A acurácia de treino chega a 100% na profundidade 25, enquanto a de validação estaciona em torno de 87% a partir da profundidade 16. A poda por custo-complexidade (Figura 3b) reduz a árvore de 2.054 para 899 folhas com $\alpha = 10^{-4}$ sem perder acurácia de validação (87,0% para 87,2%); acima de 10^{-3} a poda passa a custar acurácia, e com $\alpha = 10^{-2}$ sobram 8 folhas e 66,5%.

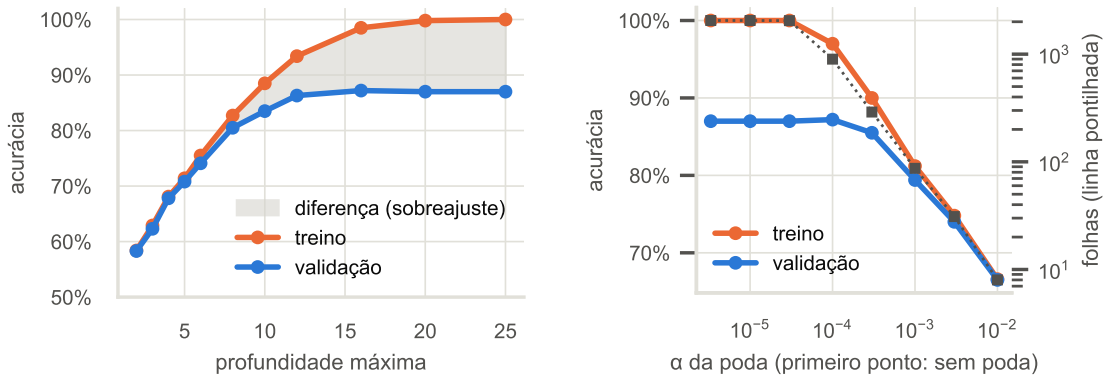


Figura 3. Acurácia de treino e de validação (seed 42, divisão agrupada, 17 atributos). (a) Por profundidade máxima. (b) Por α da poda, com o número de folhas na escala da direita (linha pontilhada); o primeiro ponto é $\alpha = 0$.

6.4. O que a árvore aprendeu

A árvore escolhida tem 1.854 folhas e 24 níveis, grande demais para ser desenhada. As primeiras camadas, porém, são legíveis (Figura 4). A raiz divide por TPSA (limiar de 67,9) e o ramo de TPSA baixa tem 69% de DRD2, como a análise exploratória antecipava. Em seguida vêm o número de átomos aromáticos, um atributo do SMILES, e depois HBD, número de anéis, comprimento do SMILES e número de halogênios. Descritores e contagens se misturam já no topo. A Figura 5 confirma o quadro: TPSA e o número de átomos aromáticos lideram nas duas medidas de importância, seguidos por HBD e por um bloco de contagens (átomos de N, halogênios, estereoquímica, anéis) acima de LogP e HBA. A importância por permutação, medida no teste, dá mais peso às contagens do que a de impureza.

6.5. Erros e limites da árvore

Entre as classes, BACE1 e DRD2 são as mais fáceis (F1 de 90,9% e 90,4%), seguidas de EGFR (89,2%) e HDAC1 (86,6%), a mais difícil. Na seed 42, o par HDAC1 $\leftrightarrow$ EGFR, nos dois sentidos, soma 28,7% dos 568 erros (Figura 6a), e 7,9% das moléculas de HDAC1 viram EGFR. Os erros não ocupam uma região do espaço de atributos: nenhum dos 17 atributos desloca a mediana entre acertos e erros em mais de meio intervalo interquartil. Entre seis exemplos de HDAC1 lidos como EGFR, dois contêm a mesma subestrutura, o grupo de átomos C(=O)NO. A árvore não a distingue: contagens de caracteres dizem quantos oxigênios e nitrogênios existem, mas não como estão ligados.

A árvore também não sabe quando erra. Sem poda, toda folha é pura, então a probabilidade de classe estimada pela árvore é 100% nas 5.496 previsões de teste, embora 10,3% estejam erradas. A curva de aprendizado (Figura 6b) mostra a outra limitação: o treino fica em 100% e a validação sobe de 61,5% (879 linhas) para 87,0% (17.584) sem se achatar, de modo que mais moléculas provavelmente renderiam mais acurácia.

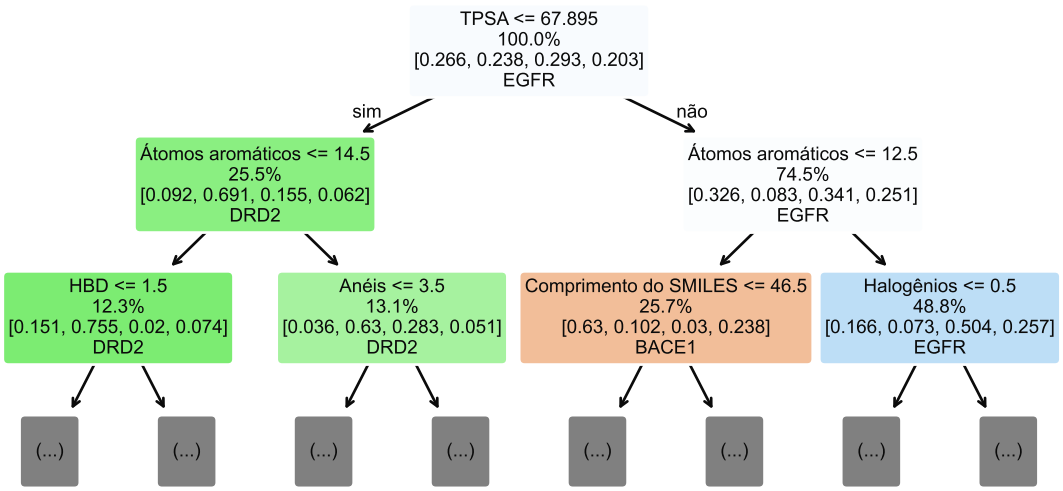


Figura 4. Três primeiras camadas da árvore escolhida (seed 42, divisão agrupada, 17 atributos). Cada nó mostra a regra de divisão, a fração das moléculas de treino que chega a ele, a proporção de cada classe (na ordem BACE1, DRD2, EGFR e HDAC1) e a classe majoritária, que também define a cor. A aresta "sim" leva às moléculas que satisfazem a regra; os nós cinza marcam subárvores omitidas.

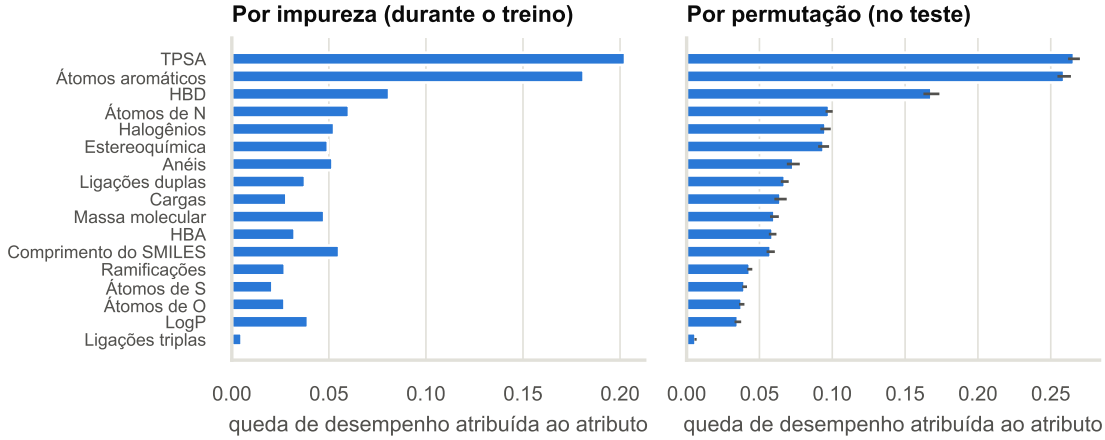


Figura 5. Importância dos atributos da árvore escolhida (seed 42). À esquerda, a redução de impureza acumulada nos nós que usam o atributo, durante o treino. À direita, a queda de acurácia no teste quando os valores do atributo são embaralhados (permutação).

6.6. Random Tree

Sozinha, a Random Tree perde para a árvore de decisão. Com os padrões do Weka, ela acerta 87,6 $\pm$ 1,1% no cenário principal, 2,0% abaixo da árvore, e fica abaixo dela em todas as 10 seeds. Com a busca em grade, chega a 89,0 $\pm$ 0,7%, ainda 0,4% abaixo, e fica abaixo da árvore em 5 dos 6 cenários (Tabela 4). A maior perda, de 2,6%, ocorre com só os cinco descritores na divisão agrupada: com 5 atributos, sortear 3 em cada nó esconde quase metade das opções de corte. O único cenário sem perda, as contagens de SMILES

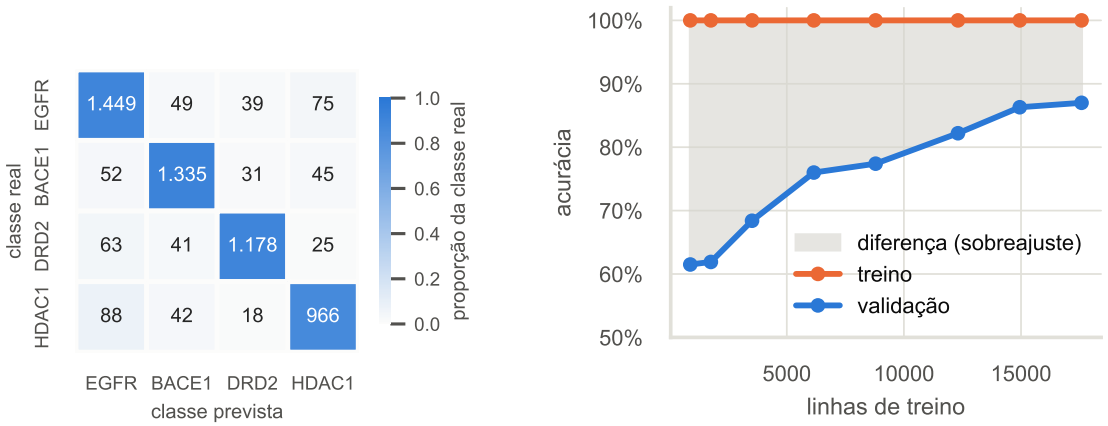


Figura 6. Seed 42, divisão agrupada, 17 atributos. (a) Matriz de confusão no teste: cor pela proporção da classe real, número em contagem. (b) Curva de aprendizado: acurácia por tamanho do conjunto de treino.

na divisão aleatória, é um empate.

Tabela 4. Random Tree ajustada contra a árvore de decisão nos seis cenários (média $\pm$ desvio em 10 seeds). Diferença: relativa, calculada em cada seed. Última coluna: em quantas das 10 seeds a Random Tree acertou mais que a árvore.

Atributos	Divisão	Árvore de decisão (%)	Random Tree (%)	Diferença (%)	Seeds com RT melhor
Descritores (5)	Aleatória	$81{,}2 \pm 0{,}5$	$80{,}5 \pm 0{,}6$	$-0{,}9$	1
Descritores (5)	Agrupada	$77{,}1 \pm 0{,}8$	$75{,}1 \pm 1{,}0$	$-2{,}6$	0
SMILES (12)	Aleatória	$88{,}9 \pm 0{,}4$	$88{,}9 \pm 0{,}6$	0,0	6
SMILES (12)	Agrupada	$87{,}6 \pm 0{,}6$	$87{,}4 \pm 0{,}7$	$-0{,}2$	4
Ambos (17)	Aleatória	$90{,}8 \pm 0{,}3$	$90{,}5 \pm 0{,}5$	$-0{,}3$	2
Ambos (17)	Agrupada	$89{,}4 \pm 0{,}4$	$89{,}0 \pm 0{,}7$	$-0{,}4$	3

A busca em grade puxa a Random Tree de volta na direção da árvore de decisão: em 5 dos 6 cenários ela escolhe sortear 75% dos atributos, a maior fração do grid. A Figura 7 mostra o mesmo efeito na seed 42. A acurácia de validação sobe de 78,7% com $K = 1$ para 86,7% com $K = 5$, o padrão do Weka, e depois fica num platô de 86,6% a 88,1%. Com $K = 17$ não há sorteio, e o modelo é a própria árvore de decisão. O sorteio também deixa o resultado instável: só trocando a semente, com os mesmos dados e a mesma divisão, a acurácia de teste da Random Tree ajustada vai de 87,8% a 89,8% em 50 sementes (desvio de 0,45 ponto), contra 88,7% a 89,4% da árvore com os mesmos hiperparâmetros (desvio de 0,17). A Random Tree aprende o mesmo que a árvore: TPSA e o número de átomos aromáticos lideram a importância, HDAC1 é a classe mais difícil e o par HDAC1 $\leftrightarrow$ EGFR concentra os erros.

6.7. Árvore única contra conjuntos de árvores

A Tabela 5 reúne todos os modelos no cenário principal. Eles formam dois grupos, separados por uma distância muito maior que o desvio entre seeds: as árvores únicas, de 87,6% a 89,4%, e a Random Forest e o *gradient boosting*, de 94,8% a 96,6%. Todos os

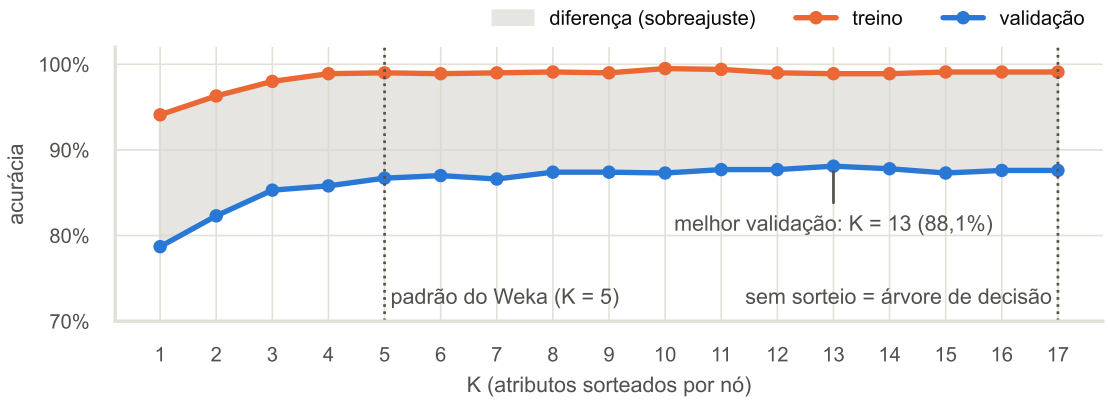


Figura 7. Random Tree: acurácia de treino e de validação agrupada por número K de atributos sorteados em cada nó (seed 42, 17 atributos, demais hiperparâmetros escolhidos pela busca). $K = 17$ desliga o sorteio e equivale à árvore de decisão.

conjuntos superam a árvore nas 10 seeds. O *gradient boosting* ajustado é o mais preciso, com 96,6 $\pm$ 0,3%, 8,1% acima da árvore.

Tabela 5. Todos os modelos na divisão agrupada, com os 17 atributos (média $\pm$ desvio em 10 seeds). Diferença para a árvore: relativa, calculada em cada seed.

Modelo	Acurácia (%)	F1 macro (%)	Diferença para a árvore (%)
Classe majoritária (EGFR)	29,3 $\pm$ 0,0	11,3 $\pm$ 0,0	–
Árvore de decisão	89,4 $\pm$ 0,4	89,3 $\pm$ 0,4	–
Random Tree (padrão do Weka)	87,6 $\pm$ 1,1	87,5 $\pm$ 1,1	−2,0
Random Tree (ajustada)	89,0 $\pm$ 0,7	88,9 $\pm$ 0,7	−0,4
Random Forest (padrão)	95,2 $\pm$ 0,3	95,1 $\pm$ 0,3	6,4
Random Forest (ajustada)	95,3 $\pm$ 0,3	95,2 $\pm$ 0,3	6,5
Gradient boosting (padrão)	94,8 $\pm$ 0,3	94,7 $\pm$ 0,3	6,0
Gradient boosting (ajustado)	96,6 $\pm$ 0,3	96,6 $\pm$ 0,2	8,1

O ajuste de hiperparâmetros pesa de forma diferente nos dois conjuntos. Na Random Forest, a floresta ajustada (95,3%) empata com a padrão (95,2%): as melhores combinações da busca ficam dentro de 0,4 ponto de validação, e só o mínimo de uma amostra por folha importa. No *boosting*, o ajuste rende 7,2 pontos sobre a árvore, contra 5,4 do padrão. Boa parte da diferença vem do que mudou junto com o grid: o padrão do scikit-learn usa 100 iterações e, com mais de 10 mil linhas, para cedo com base numa validação aleatória de 10% do treino, que aqui contém gêmeas. Com 300 iterações e sem parada antecipada, a acurácia de treino chega a 100% em 50 iterações e a de validação continua a subir até cerca de 200.

A Figura 8a (seed 42) mostra o efeito do sorteio de atributos numa árvore só e num conjunto, com a mesma árvore base. A árvore de decisão sem poda acerta 88,5%. Com sorteio ($K = \sqrt{17}$), a árvore única cai para 86,6% ($-1{,}9$ ponto), e o desvio entre sementes fica cinco vezes maior (0,87 contra 0,16 ponto): uma árvore única quer o melhor corte de

cada nó, e o sorteio às vezes o esconde. Já 300 dessas árvores, cada uma numa amostra *bootstrap* do treino, formam a Random Forest, que acerta 95,2%, 6,7 pontos acima da árvore de decisão. No conjunto, o sorteio é o que torna as árvores diferentes entre si: elas erram em moléculas diferentes, e a média cancela os erros. A Figura 8b mostra o ganho crescendo com o número de árvores: com uma ou duas, a floresta fica perto de 84%, abaixo da árvore de decisão, porque cada árvore vê só uma amostra do treino e um sorteio dos atributos; com cinco, já passa de 91%, e a partir de 50 árvores fica estável, perto de 94%. É por isso que a Random Tree perde sozinha e, ainda assim, é a peça da Random Forest.

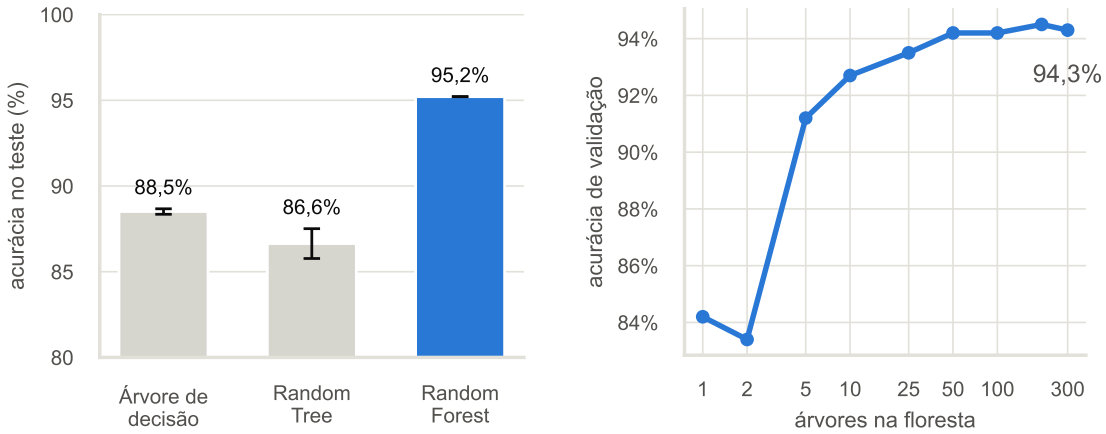


Figura 8. Da árvore à Random Forest (seed 42, divisão agrupada, 17 atributos). (a) Acurácia no teste da árvore de decisão sem poda, da Random Tree ($K = \sqrt{17}$) e da Random Forest de 300 dessas árvores; as árvores únicas mostram a média e o desvio de 10 sementes; o eixo começa em 80%. (b) Acurácia de validação agrupada da Random Forest por número de árvores.

Os conjuntos melhoram todas as classes, e HDAC1 é a que mais ganha: o F1 sobe de 86,6% na árvore para 95,4% no *boosting* ajustado. A confusão de fundo, porém, continua: o par HDAC1 $\leftrightarrow$ EGFR soma cerca de 30% dos erros em todos os modelos, que caem de 568 na árvore para 279 na Random Forest e 191 no *boosting* (seed 42). Nenhum modelo enxerga a subestrutura que as contagens não representam.

A confiança dos modelos também difere. A árvore e a Random Tree dizem 100% em quase tudo. A Random Forest sabe quando está em dúvida: as previsões com confiança abaixo de 70% são 19,8% do teste, mas reúnem 91% dos erros, e acima de 90% de confiança ela não erra. O *boosting* é confiante demais: 89,2% das previsões saem com mais de 99,9% de confiança, e essa faixa ainda guarda 20% dos erros. Para separar as previsões que merecem revisão, a Random Forest é a mais útil, apesar de acertar 1,5 ponto a menos que o *boosting*. Em todos os casos, o ganho de acurácia custa a legibilidade: só a árvore de decisão responde, com um caminho de perguntas, por que uma molécula foi classificada como DRD2.

7. Redes neurais na aprovação de empréstimos

Nesta parte do trabalho a base muda: em vez de moléculas, pedidos de empréstimo. A tarefa é prever se um pedido foi aprovado, e o modelo principal é uma rede neural do

tipo perceptron de múltiplas camadas (MLP), comparada com os modelos das seções anteriores. Além da acurácia, registramos o tempo de treino de cada modelo.

7.1. Dados e curadoria

A base é o *Loan-Approval-Prediction-Dataset*, publicado no Kaggle [architsharma01 2023]: 4.269 pedidos, cada um com um identificador, 11 atributos e o rótulo (aprovado ou rejeitado). A Tabela 6 lista os atributos com os nomes usados neste trabalho. A página não informa a moeda; como o score é o do CIBIL, o birô de crédito da Índia, os valores devem estar em rúpias. Isso não afeta os modelos, que só comparam valores entre si.

Tabela 6. Atributos da base de empréstimos: nome no arquivo original, nome usado neste trabalho e faixa de valores. Valores em dinheiro em milhões, na moeda da base.

Coluna original	Nome neste trabalho	Valores
`no_of_dependents`	dependentes	0 a 5
`education`	graduado	1 = *Graduate*, 0 = *Not Graduate*
`self_employed`	autônomo	1 = *Yes*, 0 = *No*
`income_annum`	renda anual	0,2 a 9,9
`loan_amount`	valor do empréstimo	0,3 a 39,5
`loan_term`	prazo (anos)	2 a 20, de 2 em 2
`cibil_score`	score de crédito	300 a 900
`residential_assets_value`	patrimônio residencial	0 a 29,1
`commercial_assets_value`	patrimônio comercial	0 a 19,4
`luxury_assets_value`	bens de luxo	0,3 a 39,2
`bank_asset_value`	saldo bancário	0 a 14,7
`loan_status`	aprovado (rótulo)	1 = *Approved*, 0 = *Rejected*

A curadoria foi curta, porque a base não tem valores ausentes nem linhas repetidas. O arquivo baixado é conferido pelo seu *hash* SHA-256, para garantir que todos os experimentos partem do mesmo arquivo. Os nomes de colunas e os textos vinham com espaços no início, que foram removidos. 28 pedidos tinham patrimônio residencial de -100 mil, um valor impossível. Como a taxa de aprovação nessas linhas (57%) é próxima da geral (62,2%), o valor parece um erro de digitação, e não um sinal sobre o pedido; as linhas foram mantidas e o valor foi trocado por 0, que já aparece em outros 45 pedidos. Por fim, os três textos (escolaridade, autônomo e situação) viraram 0 e 1.

7.2. Análise exploratória

62,2% dos pedidos foram aprovados; responder sempre "aprovado" é o piso desta parte. Um atributo domina os demais: o score de crédito tem correlação de 0,77 com a aprovação, enquanto o prazo tem $-0{,}11$ e os outros nove ficam perto de zero. Os valores em dinheiro são redundantes entre si: a renda anual tem correlação de 0,93 com o valor do empréstimo e de 0,93 com os bens de luxo.

A Figura 9 mostra a forma dessa relação, e ela não é gradual: abaixo de 550 pontos, cerca de 10% dos pedidos são aprovados, em qualquer faixa; a partir de 550, 99,5%. A regra "aprovar se o score for de pelo menos 550" acerta 95,4% da base. Os aprovados abaixo de 550 têm todos prazo de 2 ou 4 anos (48% e 57% de aprovação nesses

prazos); com prazos maiores, nenhum pedido abaixo de 550 foi aprovado. Os rótulos seguem, portanto, quase uma regra explícita: um degrau no score, com uma exceção para prazos curtos.

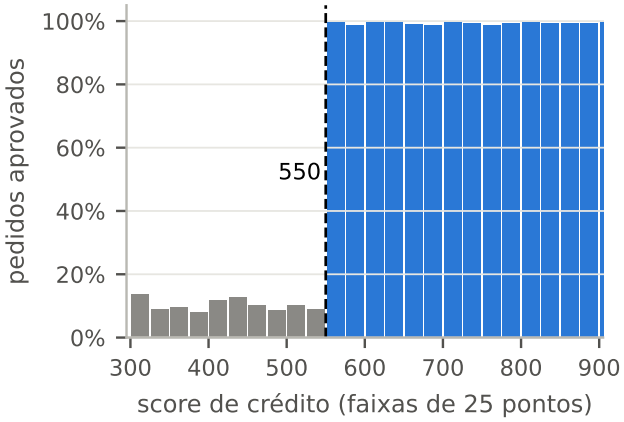


Figura 9. Proporção de pedidos aprovados por faixa de 25 pontos de score de crédito. A linha tracejada marca 550.

A base parece sintética. O score e a renda se distribuem de forma quase uniforme: os quartis do score são 453, 600 e 748, contra 450, 600 e 750 de uma distribuição uniforme entre 300 e 900. Escolaridade e autônomo se dividem quase meio a meio. Com dados de crédito reais, seriam esperadas distribuições concentradas e uma relação mais gradual entre o score e a aprovação. Isso limita o alcance das conclusões (Seção 8).

7.3. Metodologia

Protocolo. Os dados são divididos em treino (80%, 3.415 pedidos) e teste (20%, 854), mantendo a proporção de aprovados, e o procedimento é repetido com 10 seeds (0 a 9), que mudam a divisão, as partes da validação cruzada e a semente dos modelos. Não há gêmeas como na base de moléculas, então a divisão é estratificada simples. Quando um modelo tem hiperparâmetros a escolher, uma busca em grade com validação cruzada estratificada de 5 partes, feita só dentro do treino, escolhe a combinação de maior acurácia. No teste, reportam-se acurácia, F1 macro e a área sob a curva ROC (AUC). A AUC usa a probabilidade prevista, e não a classe: é a chance de um pedido aprovado, sorteado ao acaso, receber probabilidade maior que um rejeitado. Ela mede, portanto, se o modelo ordena bem os pedidos, independentemente do corte em 50%.

Modelos de referência. Cinco modelos servem de régua: a classe majoritária; a *regra de um corte*, uma árvore de profundidade 1 que escolhe sozinha, no treino, o melhor atributo e o melhor limiar (escolheu o score e 549,5 nas 10 seeds); a regressão logística com padronização, que equivale a uma rede de um único neurônio; a árvore de decisão com busca em grade (critério, profundidade máxima e mínimo por folha, 30 combinações); e a Random Forest com 300 árvores.

A rede neural. A rede é um MLP do scikit-learn [Pedregosa et al. 2011]: camadas ocultas com ativação ReLU [Nair and Hinton 2010], um neurônio de saída com sigmoide, que dá a probabilidade de aprovação, perda de entropia cruzada e otimizador Adam [Kingma and Ba 2015]. Antes da rede, cada atributo é padronizado (subtrai-se a

média e divide-se pelo desvio padrão do treino, nunca do teste). O treino usa parada antecipada [Prechelt 1998]: 10% do treino ficam de fora como validação, e o treino para quando a acurácia nessa validação passa um número fixo de épocas, a *paciência*, sem melhorar. Usamos paciência de 30 épocas em vez das 10 do padrão do scikit-learn, pelo motivo mostrado na Seção 7.4. A busca em grade cobre a arquitetura (8, 16, 32 ou 64 neurônios numa camada; 32 e 16 ou 64 e 32 em duas), a regularização L2 (α de 10^{-4} ou 10^{-2}) e a taxa de aprendizado (10^{-3} ou 10^{-2}), 24 combinações.

Tempo de execução. Para comparar algoritmos, e não o número de núcleos que cada um usa, tudo roda num único núcleo (`n_jobs=1` e as bibliotecas de álgebra linear limitadas a uma *thread*). Antes da primeira medição, uma rodada de aquecimento é feita e descartada. Nos modelos com busca em grade, o tempo da busca é medido à parte, e o modelo final é treinado de novo com os hiperparâmetros escolhidos para medir só o treino. O resumo usa a mediana das 10 seeds. A máquina é um AMD Ryzen 5 9600X (6 núcleos) com 31 GB de memória, Windows 11, Python 3.14 e scikit-learn 1.9.1, sem outras tarefas pesadas durante a execução.

7.4. Resultados

Padronização. A mesma rede, com 16 neurônios numa camada, acerta 59,5 $\pm$ 3,1% sem padronização, abaixo do piso de 62,2%, com AUC de 51,0%, o mesmo que sortear. Com padronização, acerta 93,9 $\pm$ 0,7%. Sem ela, a renda e os patrimônios, na casa dos milhões, dominam a soma ponderada de cada neurônio, e o passo do otimizador, do mesmo tamanho para todos os pesos, não serve ao mesmo tempo para atributos de 0 a 5 e de 0 a 40 milhões [LeCun et al. 1998]. A árvore não sofre com isso, porque compara cada atributo apenas com um limiar.

Neurônios. A Figura 10 mostra a acurácia de validação cruzada (seed 42) por número de neurônios. Com 1 neurônio por camada, a rede fica perto da regressão logística (93,4% com uma camada). Com 2 a 8 neurônios, já passa de 97%, e o melhor ponto, duas camadas de 8, chega a 98,0%, perto da árvore. Mais neurônios não ajudam: com 64 numa camada, a acurácia cai para 95,4%. A curva é irregular porque cada ponto é uma única rede, e o resultado depende dos pesos iniciais sorteados.

Épocas e paciência. Cada época é uma passada completa pelo treino. A Figura 11 acompanha uma rede de 32 neurônios por 400 épocas, sem parada antecipada. A perda de treino continua caindo, enquanto a de validação se estabiliza: a partir daí, a rede passa a ajustar detalhes do treino que não se repetem na validação (Figura 11a). Com taxa de aprendizado de 10^{-3}, a rede aprende devagar, e a acurácia de validação passa longos trechos sem melhorar. Com paciência 10, o treino pararia na época 37, com 92,1% de validação, quando a mesma rede chegaria a 96,5%; com paciência 30, para na época 217, com 95,5% (Figura 11b). Com taxa de 10^{-2}, chega a 97% em menos de 100 épocas. A busca em grade confirmou o quadro: escolheu a taxa de 10^{-2} em 10 das 10 seeds e duas camadas ocultas em 9 (32 e 16 neurônios em 7), e a rede final treinou de 40 a 121 épocas.

Comparação com os modelos de referência. A Tabela 7 reúne os seis modelos. A rede acerta 96,4 $\pm$ 1,1%. Comparando seed a seed, ela supera a regressão logística em todas as 10 seeds (4,5 pontos), o que mostra o ganho das camadas ocultas sobre um neurônio só, e supera a regra de um corte em 8 das 10 (0,9 ponto, menos que o desvio entre seeds). Perde para a árvore de decisão em todas as seeds ($-$1,7 ponto) e para a

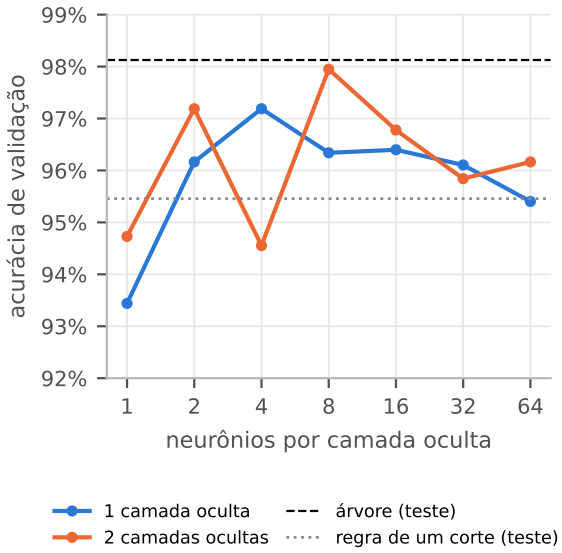


Figura 10. Acurácia de validação cruzada (5 partes, seed 42) por número de neurônios em cada camada oculta, com taxa de aprendizado de 10^{-2} e sem parada antecipada. As linhas horizontais são a acurácia de teste da árvore e da regra de um corte (média de 10 seeds).

Random Forest em 9 ($-1{,}9$ ponto). Também é o modelo menos estável: o desvio de 1,1 ponto é quase três vezes o da árvore. A AUC conta outra parte da história. A da rede (99,3%) é maior que a da árvore (98,7%), assim como a da regressão logística (97,0%) é maior que a da regra de um corte (96,2%), que acerta mais: os modelos de saída suave ordenam bem os pedidos, e o que erram é o corte em 50%. Árvore e Random Forest empatam (0,2 ponto, a floresta melhor em 5 das 10 seeds).

Tabela 7. Os seis modelos na base de empréstimos (média $\pm$ desvio em 10 seeds; erros em 854 pedidos de teste). Tempos: mediana das 10 seeds, num único núcleo. Treino: o do modelo final; busca: a busca em grade com validação cruzada de 5 partes; previsão: os 854 pedidos de teste.

Modelo	Acurácia (%)	F1 macro (%)	AUC (%)	Erros	Treino (ms)	Busca (s)	Previsão (ms)
Classe majoritária	62,2 $\pm$ 0,0	38,3	50,0	323,0	0,2	–	0,03
Regra de um corte	95,5 $\pm$ 0,6	95,3	96,2	38,8	1,9	–	0,3
Regressão logística	91,9 $\pm$ 1,1	91,3	97,0	69,5	3,5	–	0,5
Árvore de decisão	98,1 $\pm$ 0,4	98,0	98,7	16,0	3,7	0,65	0,3
Random Forest	98,3 $\pm$ 0,4	98,2	99,8	14,5	490	–	15,3
Rede neural (MLP)	96,4 $\pm$ 1,1	96,1	99,3	30,9	183	23,5	0,6

Tempo. Treinar a rede final leva 183 ms, cerca de 50 vezes os 3,7 ms da árvore, e menos da metade dos 490 ms da Random Forest. O maior custo da rede está na escolha dos hiperparâmetros: a busca leva 23,5 s por seed, contra 0,65 s da árvore (36 vezes mais), porque cada uma das 24 combinações treina 5 redes por dezenas de épocas. Prever é rápido para quase todos: 0,6 ms para a rede e menos de 1 ms para a árvore, contra 15 ms da Random Forest, que consulta 300 árvores.

Onde a rede erra. Na seed 42, a rede comete 23 erros no teste, e a árvore, 15. Os

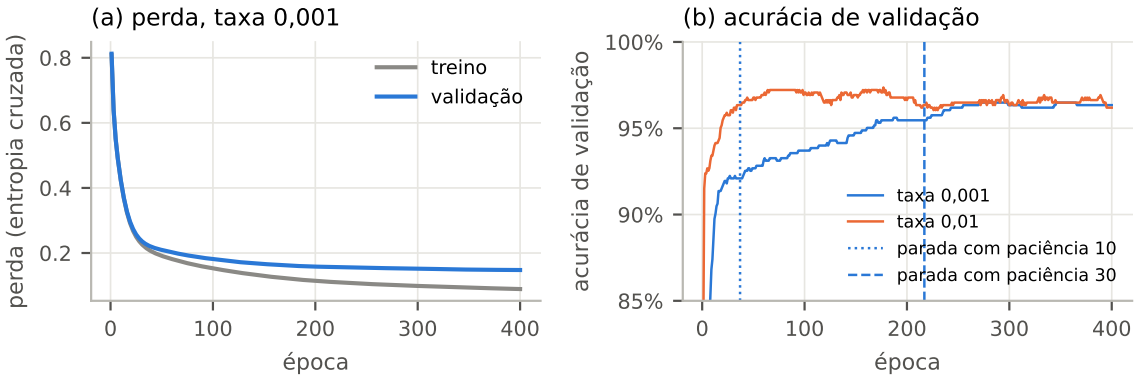


Figura 11. Uma rede de 32 neurônios treinada por 400 épocas (seed 42; 80% do treino para ajustar e 20% para validar). (a) Perda no treino e na validação, com taxa de aprendizado de 10^{-3}. (b) Acurácia de validação com as duas taxas; as linhas verticais marcam onde a parada antecipada interromperia a rede de taxa 10^{-3} com paciência de 10 e de 30 épocas.

erros da rede se concentram logo acima do degrau: entre 550 e 599 pontos há 73 pedidos de teste, e a rede erra 9, contra 1 da árvore. A árvore, por sua vez, erra sobretudo na exceção dos prazos curtos (score abaixo de 550 e prazo de até 4 anos, 52 pedidos): 12 dos seus 15 erros, contra 8 da rede. A Figura 12 mostra a causa. A árvore representa o degrau com uma pergunta e dá probabilidades de 0 ou 100% em quase todos os pedidos. A rede desenha uma rampa: entre 500 e 599 pontos, 24% das suas probabilidades ficam entre 10% e 90%, contra 1% na árvore, e é nessa rampa que estão os seus erros. Redes neurais tendem a aprender funções suaves, e uma relação em degrau é justamente o caso em que modelos de árvore costumam levar vantagem em dados tabulares [Grinsztajn et al. 2022].

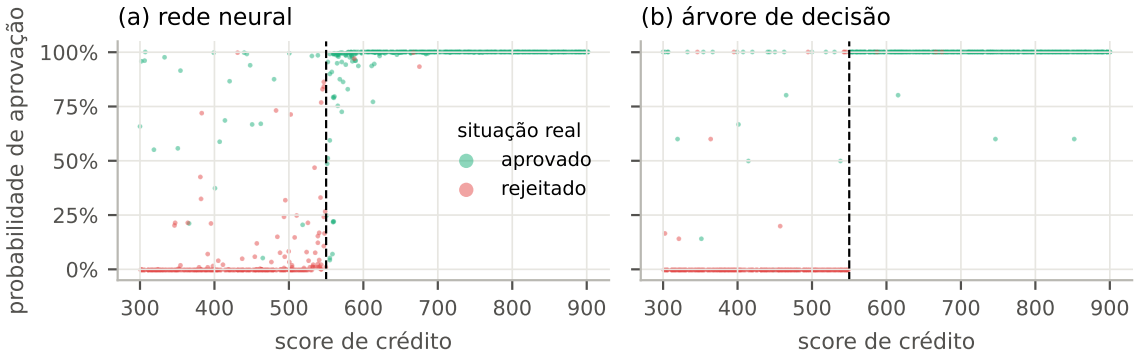


Figura 12. Probabilidade de aprovação prevista para cada pedido de teste, por score de crédito (seed 42), colorida pela situação real. (a) Rede neural. (b) Árvore de decisão. A linha tracejada marca 550.

8. Limitações

O agrupamento por descritores idênticos não cobre famílias de moléculas muito parecidas entre si, que continuam espalhadas entre treino e teste; as contagens de SMILES permitem decorar essas famílias. Dividir por família (esqueleto molecular) controlaria isso, mas exige uma biblioteca de química. Por isso a acurácia de 89,4% ainda é otimista para moléculas realmente novas.

O modelo escolhe sempre uma entre quatro classes: não existe a opção "nenhuma das quatro", então ele não sabe dizer que uma molécula não pertence a nenhum dos alvos. As 10 seeds variam o sorteio da divisão, mas os dados são os mesmos, de modo que o desvio padrão não mede a incerteza para dados novos, e não há validação externa. Por fim, as 238 moléculas sem descritores (0,86%) foram descartadas, em sua maioria muito grandes, e as probabilidades da árvore não são calibradas.

Na comparação entre modelos, a busca da Random Forest e do *gradient boosting* foi leve e feita só no cenário principal, de modo que o teto pode estar um pouco subestimado. A comparação entre as três árvores base e as curvas usam uma única seed. A Random Tree foi reproduzida no scikit-learn a partir da descrição do algoritmo do Weka, sem rodar o próprio Weka, e pode diferir dele em detalhes como o tratamento de empates. Os tempos de treino não são reportados porque foram medidos com a máquina ocupada por outros processos.

Na base de empréstimos, a principal limitação é a própria base. Distribuições quase uniformes e um rótulo que segue um degrau no score indicam dados sintéticos, gerados por uma regra; em dados de crédito reais, a relação entre os atributos e a aprovação tende a ser mais gradual, o que poderia favorecer a rede. A conclusão de que a árvore vence a rede vale, portanto, para esta base, e não para crédito em geral. A busca da rede cobriu 24 combinações e só o otimizador Adam, e a semente dos pesos iniciais mudou junto com a divisão dos dados, de modo que o desvio entre seeds mistura as duas fontes de variação. Os tempos foram medidos numa única máquina, num núcleo só; os valores absolutos mudam com o computador, e as razões entre modelos são mais estáveis que os tempos em si.

9. Conclusão e trabalhos futuros

Reconstruímos do ChEMBL_37 uma base de 27.714 moléculas, cada uma associada a um de quatro alvos (EGFR, BACE1, DRD2 e HDAC1), e treinamos uma árvore de decisão para prever o alvo. Com divisão agrupada, os cinco descritores físico-químicos dão acurácia de 77,1%; acrescentar 12 contagens de caracteres do SMILES leva a 89,4 $\pm$ 0,4% (ganho relativo de 16%), e as contagens sozinhas já chegam a 87,6%. A divisão aleatória superestima o resultado em 1,5% a 5,3% (relativo) porque cerca de um quarto das moléculas de teste tem gêmea de descritores no treino. TPSA e o número de átomos aromáticos são os atributos mais importantes, e o par HDAC1–EGFR concentra 28,7% dos erros.

Na comparação com a Random Tree e com conjuntos de árvores, a árvore de decisão segue como a melhor árvore única. A Random Tree fica 0,4% abaixo dela quando ajustada e 2,0% abaixo com os padrões do Weka, e varia mais com a semente: o sorteio de atributos esconde o melhor corte de cada nó, e uma árvore só não tem como compensar. Nos conjuntos, o mesmo sorteio ajuda: torna as árvores diferentes entre si, e 300 Random Trees juntas, a Random Forest, acertam 6,7 pontos a mais que a árvore de decisão na seed 42. O *gradient boosting* ajustado é o modelo mais preciso (96,6%, 8,1% acima da árvore). A Random Forest chega a 95,2% sem ajuste e é a única cuja confiança separa acertos de erros. Em troca, só a árvore de decisão oferece regras legíveis.

Na base de 4.269 pedidos de empréstimo, uma rede neural com padronização e parada antecipada acerta 96,4 $\pm$ 1,1%: supera a regressão logística, um neurônio só, nas 10 seeds, e a regra de um corte em 8 delas. Sem padronização, ela não aprende

(59,5%). Com a paciência padrão de 10 épocas e taxa de aprendizado de 10^{-3}, pararia cedo demais (92,1% de validação). Mesmo bem ajustada, a rede perde para a árvore de decisão (98,1%) em todas as seeds, treina cerca de 50 vezes mais devagar e exige uma busca de hiperparâmetros 36 vezes mais longa. O motivo é a forma da regra que gerou os rótulos, um degrau no score de crédito: a árvore o representa com uma pergunta, e a rede só o aproxima com uma rampa, onde ficam seus erros. Nesta base, a rede funciona, mas não compensa.

Quatro caminhos seguem do que foi visto. Representações estruturais da molécula (*fingerprints*), calculadas com uma biblioteca de química, enxergariam subestruturas como o grupo C(=O)NO, que as contagens não veem. Uma divisão por esqueleto molecular daria uma estimativa menos otimista para moléculas novas. A curva de aprendizado ainda sobe, o que sugere buscar mais dados no ChEMBL, com outros alvos ou um limiar de pChEMBL mais baixo. Por fim, calibrar as probabilidades do *gradient boosting*, hoje confiante demais, permitiria usar a confiança do modelo mais preciso para escolher quais previsões revisar. Na base de empréstimos, o próximo passo seria repetir a comparação em dados de crédito reais, onde a relação com o score é gradual, e separar a variação causada pelos pesos iniciais da rede da variação causada pela divisão dos dados.

Referências

architsharma01 (2023). Loan-Approval-Prediction-Dataset. Kaggle, versão 1, licença MIT. `https://www.kaggle.com/datasets/architsharma01/loan-approval-prediction-dataset`.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group, Belmont, CA.

Davies, M., Nowotka, M., Papadatos, G., Dedman, N., Gaulton, A., Atkinson, F., Bellis, L., and Overington, J. P. (2015). ChEMBL web services: streamlining access to drug discovery data and utilities. *Nucleic Acids Research*, 43(W1):W612–W620.

Ertl, P., Rohde, B., and Selzer, P. (2000). Fast calculation of molecular polar surface area as a sum of fragment-based contributions and its application to the prediction of drug transport properties. *Journal of Medicinal Chemistry*, 43:3714–3717.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.

Gaulton, A. et al. (2012). ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic Acids Research*, 40(D1):D1100–D1107.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.

Kapoor, S. and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154.

Keiser, M. J., Roth, B. L., Armbruster, B. N., Ernsberger, P., Irwin, J. J., and Shoichet, B. K. (2007). Relating protein pharmacology by ligand chemistry. *Nature Biotechnology*, 25(2):197–206.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

LeCun, Y., Bottou, L., Orr, G. B., and Müller, K.-R. (1998). Efficient BackProp. In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 9–50. Springer.

Lipinski, C. A., Lombardo, F., Dominy, B. W., and Feeney, P. J. (2001). Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 46(1–3):3–26.

Nair, V. and Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Prechelt, L. (1998). Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.

Sheridan, R. P. (2013). Time-split cross-validation as a method for estimating the goodness of prospective prediction. *Journal of Chemical Information and Modeling*, 53(4):783–790.

Weininger, D. (1988). SMILES, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36.

Witten, I. H., Frank, E., Hall, M. A., and Pal, C. J. (2016). *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, 4 edition.

xjoannax88 (2026). MultiTarget Bioactivity ChEMBL. Kaggle, versão 17. `https://www.kaggle.com/datasets/xjoannax88/multitarget-bioactivity-chembl`.

Zdrazil, B. et al. (2024). The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192.